Pattern Recognition Letters 45 (2014) 99–105

Contents lists available at ScienceDirect

Pattern Recognition Letters

journal homepage: www.elsevier.com/locate/patrec

# Using derivatives in a longest common subsequence dissimilarity measure for time series classification ☆

Tomasz Górecki *

*Adam Mickiewicz University, Umultowska 87, 61-614 Poznań, Poland*

ARTICLE INFO

*Article history:*
Received 12 November 2013
Available online 27 March 2014

*Keywords:*
Longest common subsequence
Derivative longest common subsequence
Data mining
Time series

ABSTRACT

Over recent years the popularity of time series has soared. Given the widespread use of modern information technology, a large number of time series may be collected. As a consequence there has been a dramatic increase in the amount of interest in querying and mining such data. A vital component in many types of time series analyses is the choice of an appropriate dissimilarity measure. Numerous measures have been proposed to date, with the most successful ones based on dynamic programming. One of such measures is longest common subsequence (LCSS). In this paper, we propose a parametrical extension of LCSS based on derivatives. In contrast to well-known measures from the literature, our approach considers the general shape of a time series rather than point-to-point function comparison. The new dissimilarity measure is used in classification with the nearest neighbor rule. In order to provide a comprehensive comparison, we conducted a set of experiments, testing effectiveness on 47 real time series. Experiments show that our method provides a higher quality of classification compared with LCSS on examined data sets.



## 1. Introduction

Time series classification has been studied extensively by the machine learning and data mining communities. Such series are suitable for representing social, economic and natural phenomena, medical observations, and results of scientific and engineering experiments. The crucial point in time series classification is how to measure the dissimilarity of time series (a very good overview of dissimilarity measures can be found in [9]). The simplicity and efficiency of Euclidean distance [11] makes this the most popular dissimilarity measure in time series data mining [1,18]. It requires that both input sequences be of the same length, and it is sensitive to distortions and shifting along the time axis [29,25]. Such problems can be handled by elastic dissimilarity measures such as Dynamic Time Warping (DTW) [5] and Longest Common SubSequence (LCSS) [2,28]. DTW searches for the best alignment between two time series, attempting to minimize the distance between them. LCSS finds the length of the longest matching subsequence. Compared with Euclidean distance, DTW and LCSS are more elastic, supporting local time shifts and variations in lengths of pairs of time series, but they are also more expensive to compute. Of the three measures, LCSS is the least sensitive to noise, because it includes a threshold to define a "match" [28].

The effectiveness of the nearest neighbor classifier depends on the dissimilarity measure used to compare objects in the classification process. At present, the dissimilarity functions used in time series classification mostly involve point-to-point comparison of time series. The measures often reduce such distortions as occur if two time series do not have the same length or are locally out of phase, etc. It seems that in the classification domain there could be objects for which function value comparison is not sufficient. There could be cases where assignment to one of the classes depends on the general shape of objects rather than on strict function value comparison. An object associated with a function that responds to its variability in "time" is the derivative of the function. The function's derivative determines areas where the function is constant, increases or decreases, and the intensity of the changes. The derivative determines the general shape of the function rather than the value of the function at a particular point. The derivative shows what happens in the neighborhood of the point. While the first derivative gives some information about the shape of the function (increasing or decreasing), the second derivative adds additional information as to where the function is convex or concave. We cannot expect that it will be sufficient to compare only time series derivatives. It seems that the best approach is to create a method which considers both the function values of time series

☆ This paper has been recommended for acceptance by A. Marcelli.

* Tel.: +48 618295308.
*E-mail address:* tomasz.gorecki@amu.edu.pl

http://dx.doi.org/10.1016/j.patrec.2014.03.009
0167-8655/

and values of the derivative (or derivatives) of the function (shape comparison). The intensity of the influence of these approaches should be parameterized. Then we can expect that for different time series the method will select the appropriate intensities of these comparisons and give the best classification results.

In this paper we construct a dissimilarity measure that considers the above-mentioned approaches to time series classification. Consequently we are able to deal with situations where the investigated sequences are not different enough. For a dissimilarity function, a new parameterized family of dissimilarity measures is formed, where a fixed dissimilarity measure is used to compute dissimilarities of time series (function values) and their variability in "time" (dissimilarities of their derivatives). The new dissimilarity functions so constructed are used in the nearest neighbor classification method. The use of derivatives in time series classification is not a novelty. Some ideas of dissimilarity between trajectories using derivatives were proposed by Kosmelj [20] and Carlier [6]. They used the concepts of velocity and acceleration to measure the dissimilarities between trajectories in cluster analysis. D'Urso and Vichi [10] and Coppi et al. [7] developed this idea and used it to perform cluster analysis of longitudinal data. The use of derivatives with DTW was proposed by Keogh and Pazzani [17]. However they used only the dissimilarity between the derivatives, rather than the standard dissimilarity between the time series. Górecki [13] and Górecki and Łuczak [14] presented results concerning derivative DTW where just the first derivative is added, while parameterization involves both the function and derivative. Such an approach was shown to give very good results. Górecki and Łuczak [15] also presented results where the second derivative is added. The parametric approach makes it possible to adapt to the data set, but without overtraining. Now we try to extend this methodology to LCSS, which is a better method than DTW in the presence of outliers [28] and generally is very close to the best dissimilarity measure DTW [21].

In this paper we first review the concept of time series and the longest common subsequence dissimilarity measure (Section 2). At the end of that section we introduce our dissimilarity measure based on derivatives. The data sets used and the experimental setup are described in Section 3. Section 4 contains the results of our experiments on the described real data sets, as well as statistical analysis of the results and analysis of the running times of the investigated methods. Conclusions are given in Section 5.

## 2. Methods

### 2.1. Longest common subsequence

The longest common subsequence dissimilarity measure is a variation of the edit dissimilarity measure used in speech recognition. The basic idea is to match two sequences by allowing them to stretch, without rearranging the sequence of the elements but allowing some elements to be unmatched or left out (e.g., outliers) – whereas in Euclidean Distance and DTW, all elements from both sequences must be used, even the outliers. The overall idea is to count the number of pairs of points from the two sequences that match. One point can never be associated twice with points of the other sequence, so that the maximum number of associations is the smaller length of the two sequences. The LCSS measure has two parameters, $\delta$ and $\varepsilon$ (Fig. 1). The constant $\delta$, which is usually set to a percentage of the sequence length, is a warping threshold and controls the window size for matching a given point from one sequence to a point in another sequence. It controls how far in time we can go in order to match a given point from one trajectory to a point in another trajectory. The constant $0 < \varepsilon < 1$ is the matching threshold: two points from two sequences are considered to match if their distance is less than $\varepsilon$.

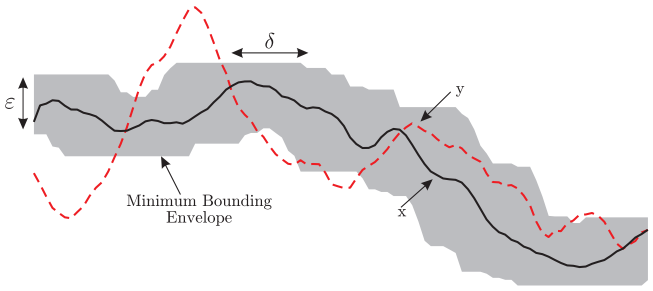


**Fig. 1.** Matching within $\delta$ in time and $\varepsilon$ in space. Everything outside the bounding envelope can never be matched.

Longest common subsequences of the time series $x$ and $y$ of length $n$ and $m$ may be recursively defined as follows:

$$L(i,j) = \begin{cases} 0 & \text{for } i = 0 \\ 0 & \text{for } j = 0 \\ 1 + L[i-1, j-1] & \text{for } |x_i - y_j| < \varepsilon \text{ and } |i-j| \leqslant \delta \\ \max(L[i-1,j], L[i,j-1]) & \text{in other cases} \end{cases}$$

$L(n,m)$ contains the similarity between $x$ and $y$, because it corresponds to the length of the longest common subsequence of elements between time series $x$ and $y$. To define the dissimilarity between $x$ and $y$, we can compute [24]:

$$\mathrm{LCSS}(x,y) = \frac{n+m-2L(n,m)}{n+m}. \quad (1)$$

According to this definition, this measure takes values from $(1 - 2\frac{\min(n,m)}{n+m})$ to 1. For two trajectories of equal length it takes values from 0 to 1.

Taking into account only sufficiently similar points, LCSS solves the problem of the presence of noise, but does not satisfy the triangle inequality [28], so it is not a distance metric. LCSS is robust to noise and is expected to be more accurate than DTW in the presence of outliers.

### 2.2. Longest common subsequence based on derivatives

Let LCSS be the longest common subsequence dissimilarity measure for two time series $x$ and $y$.

#### 2.2.1. 2D method

A dissimilarity measure which considers both the function values of time series and values of the first derivative is defined by:

$$\mathrm{DD}_{\mathrm{LCSS}}(x,y) := a\,\mathrm{LCSS}(x,y) + b\,\mathrm{LCSS}(\nabla x, \nabla y), \quad (2)$$

where $\nabla x$ and $\nabla y$ are the first discrete derivatives of $x, y$, and $a, b \in [0,1]$ are parameters. The discrete derivative of a time series $x$ with length $n$ is defined by

$$\nabla x(i) = x(i+1) - x(i), \quad i = 1, 2, \ldots, n-1. \quad (3)$$

#### 2.2.2. 3D method

A dissimilarity measure which considers both the function values of time series and values of the first and second derivatives is defined by:

$$2\mathrm{DD}_{\mathrm{LCSS}}(x,y) := a\,\mathrm{LCSS}(x,y) + b\,\mathrm{LCSS}(\nabla x, \nabla y) + c\,\mathrm{LCSS}(\nabla^2 x, \nabla^2 y), \quad (4)$$

where $\nabla^2 x$ and $\nabla^2 y$ are the second discrete derivatives of $x, y$, and $a, b, c \in [0,1]$ are parameters.

If the similarity measure in the above definitions is a metric, then the new measures $\mathrm{DD}_{\mathrm{LCSS}}$ and $2\mathrm{DD}_{\mathrm{LCSS}}$ are also metrics. In

our case the base similarity measure is LCSS, which is not a distance metric, so $DD_{LCSS}$ and $2DD_{LCSS}$ are not metrics.

### 2.3. Parameter dimension

#### 2.3.1. 2D method

We do not have to check all values of $a, b \in [0,1]$. If $a_1 = ka_2$ and $b_1 = kb_2$, where $k > 0$ is a constant (i.e. points $(a_1, b_1)$, $(a_2, b_2)$ are linearly dependent), we have

$$DD^{a_1 b_1}_{LCSS}(x_1, y_1) \underset{>}{\overset{=}{<}} DD^{a_1 b_1}_{LCSS}(x_2, y_2) \Longleftrightarrow DD^{a_2 b_2}_{LCSS}(x_1, y_1) \underset{>}{\overset{=}{<}} DD^{a_2 b_2}_{LCSS}(x_2, y_2)$$

so we can choose points $(a, b)$ on any continuous line between the points (0, 1) and (1, 0). We take the simplest case – the straight line (Fig. 2) with equation:

$$\begin{aligned} a &= (1 - \alpha), \\ b &= \alpha, \end{aligned} \qquad \alpha \in [0,1].$$

If the subset of the parameters $\alpha$ is dense enough, the choice of parameterization should not be critical.

#### 2.3.2. 3D method

We do not have to check all values of $a, b, c \in [0,1]$. If $a_1 = ka_2$, $b_1 = kb_2$, $c_1 = kc_2$ where $k > 0$ is a constant (i.e., points $(a_1, b_1, c_1), (a_2, b_2, c_2)$ are linearly dependent), we have

$$2DD^{a_1 b_1 c_1}_{LCSS}(x_1, y_1) \underset{>}{\overset{=}{<}} 2DD^{a_1 b_1 c_1}_{LCSS}(x_2, y_2) \Longleftrightarrow 2DD^{a_2 b_2 c_2}_{LCSS}(x_1, y_1) \underset{>}{\overset{=}{<}} 2DD^{a_2 b_2 c_2}_{LCSS}(x_2, y_2)$$

hence we can choose points $(a, b, c)$ on any continuous surface between the points $A = (1,0,0)$, $B = (0,1,0)$, and $C = (0,0,1)$. For example, this may be the area of the triangle with vertices at those points, or one-eighth of a sphere. For simplicity, we choose the triangle. This 3D triangle is mapped to the 2D triangle with vertices at the points $A' = (0,0)$, $B' = (1,0)$, and $C' = \left(\frac{1}{2}, \frac{\sqrt{3}}{2}\right)$. Both triangles can be defined in parametrical fashion:

$$(a, b, c) = A + \alpha\overrightarrow{AB} + \beta\overrightarrow{AC},$$

$$(a', b') = A' + \alpha\overrightarrow{A'B'} + \beta\overrightarrow{A'C'},$$

where $(a, b, c)$ are points of the 3D triangle, $(a', b')$ are points of the 2D triangle, and $\alpha \in [0,1], \beta \in [0, 1 - \alpha]$ are parameters (Fig. 3).

In what follows we will use the 2D parameters $\alpha, \beta$ (or $a', b'$) instead of the 3D parameters $a, b, c$.

## 3. Experimental setup

We performed experiments on 47 data sets. Information on the time series used is presented in Table 1. The time series originate from the UCR Time Series Classification/Clustering Homepage [19], which includes the majority of all of the world's publicly available, labeled time series data sets. The length of time series varies from 24 to 1882 depending on the data set. The number of time series in the training (testing) set per data set varies from 16 (28) to 1800 (8236), and the number of classes varies from 2 to 50. The data sets originate from a plethora of different domains, including medicine, robotics, astronomy, biology, face recognition, handwriting recognition, etc.

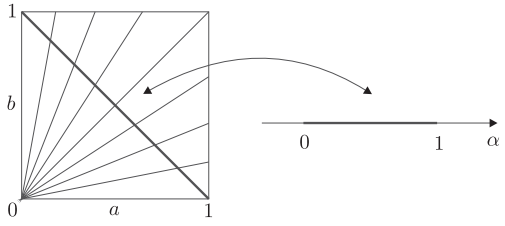


**Fig. 2.** Dependence of parameters $a$, $b$ and $\alpha$.

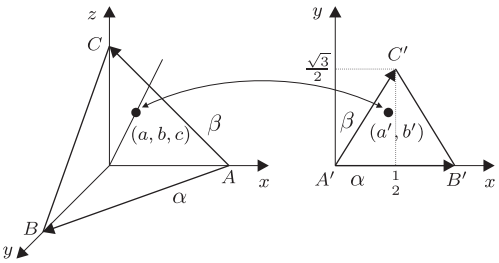


**Fig. 3.** Relationship between the 3D parameters $a$, $b$, $c$ and 2D parameters $\alpha$, $\beta$.

The proposed dissimilarity measures are used with the 1NN method in the classification process. The 1NN classifier is a very simple classifier: the class of an unclassified time series is determined to be the class of its most similar time series. This choice of classifier was motivated by reports that, despite its simplicity, it achieves results which are among the best compared with other more complex classifiers for time series [9,23,26]. For each data set we calculated the classification error rate on a test subset. We found all parameters using the training subset. An appropriate distribution of the training and test sets is proposed by the authors of the repository (each data set is divided into a training and testing subset). We use the leave-one-out cross-validation (CV) method to find the best parameter $\alpha$ (the best pair of parameters $\alpha, \beta$) in our classifier. If the minimal error rate is the same for more than one value of the parameter $\alpha$ we choose the smallest one (the smallest pair – minimizing $\beta$ first, then $\alpha$). As a result of this method of minimization, if $2DD_{LCSS}$ and $DD_{LCSS}$ have the same CV error rate, then they have the same test error rate as well. Finite subsets of values of $\alpha$ and $\beta$ are chosen, ranging from 0 to 1 with fixed step size 0.01.

The values we used for $\delta$ and $\varepsilon$ for LCSS are clearly dependent on the application and the data set. We set $\delta$ to 100%. We selected this value because LCSS is very resistant to changes in $\delta$; the average error rate does not vary with a change in the value of this parameter. Performance decreases only for values below 6%. For DTW we observe a slight increase in performance with a change in the window width. This suggests that LCSS could be a safer choice because of the less pronounced need for tuning of the $\delta$ parameter [21]. The determination of $\varepsilon$ is application-dependent. We used a value equal to the smallest standard deviation between the two trajectories that were examined at any time [28].

The PRTools 4.1.13 program (http://www.prtools.org) was used in the computational process. This is a Matlab (version R2011a) based toolbox for pattern recognition [27]. The hardware platform was a PC with Intel Core i7-2600K 3.4 GHz processor (8 GB RAM) and Windows 7 Professional SP1 (64-bit) operating system.

## 4. Results

The results are presented in Table 2. The absolute error rates on the test subset with the 1NN method are shown for each dissimilarity measure.

LCSS performed the best on 2 of the data sets, $DD_{LCSS}$ on 6 and $2DD_{LCSS}$ on 23. On 16 data sets no method was clearly better than the others. Comparing the new dissimilarity measures with the standard LCSS, we can see a significant reduction in error rate for

**Table 1**
Summary of data sets.

| Data set | Number of classes | Size of training set | Size of testing set | Time series length |
|---|---|---|---|---|
| 50Words | 50 | 450 | 455 | 270 |
| Adiac | 37 | 390 | 391 | 176 |
| Beef | 5 | 30 | 30 | 470 |
| Car | 4 | 60 | 60 | 577 |
| CBF | 3 | 30 | 900 | 128 |
| ChlorineConcentration | 3 | 467 | 3840 | 166 |
| CinC_ECG_torso | 4 | 40 | 1380 | 1639 |
| Coffee | 2 | 28 | 28 | 286 |
| Cricket_X | 12 | 390 | 390 | 300 |
| Cricket_Y | 12 | 390 | 390 | 300 |
| Cricket_Z | 12 | 390 | 390 | 300 |
| DiatomSizeReduction | 4 | 16 | 306 | 345 |
| ECG200 | 2 | 100 | 100 | 96 |
| ECGFiveDays | 2 | 23 | 861 | 136 |
| Face (all) | 14 | 560 | 1690 | 131 |
| Face (four) | 4 | 24 | 88 | 350 |
| FacesUCR | 14 | 200 | 2050 | 131 |
| Fish | 7 | 175 | 175 | 463 |
| Gun-Point | 2 | 50 | 150 | 150 |
| Haptics | 5 | 155 | 308 | 1092 |
| InlineSkate | 7 | 100 | 550 | 1882 |
| ItalyPowerDemand | 2 | 67 | 1029 | 24 |
| Lightning-2 | 2 | 60 | 61 | 637 |
| Lightning-7 | 7 | 70 | 73 | 319 |
| MALLAT | 8 | 55 | 2345 | 1024 |
| MedicalImages | 10 | 381 | 760 | 99 |
| MoteStrain | 2 | 20 | 1252 | 84 |
| Non-Invasive Thorax1 | 42 | 1800 | 1965 | 750 |
| Non-Invasive Thorax2 | 42 | 1800 | 1965 | 750 |
| OliveOil | 4 | 30 | 30 | 570 |
| OSU Leaf | 6 | 200 | 242 | 427 |
| Plane | 7 | 105 | 105 | 144 |
| SonyAIBORobot Surface | 2 | 20 | 601 | 70 |
| SonyAIBORobot SurfaceII | 2 | 27 | 953 | 65 |
| StarLightCurves | 3 | 1000 | 8236 | 1024 |
| Swedish Leaf | 15 | 500 | 625 | 128 |
| Symbols | 6 | 25 | 995 | 398 |
| Synthetic Control | 6 | 300 | 300 | 60 |
| Trace | 4 | 100 | 100 | 275 |
| Two Patterns | 4 | 1000 | 4000 | 128 |
| TwoLeadECG | 2 | 23 | 1139 | 82 |
| uWaveGestureLibrary_X | 8 | 896 | 3582 | 315 |
| uWaveGestureLibrary_Y | 8 | 896 | 3582 | 315 |
| uWaveGestureLibrary_Z | 8 | 896 | 3582 | 315 |
| Wafer | 2 | 1000 | 6174 | 152 |
| WordsSynonyms | 25 | 267 | 638 | 270 |
| Yoga | 2 | 300 | 3000 | 426 |

**Table 2**
Testing error rates (in %).

| Data set | LCSS | $DD_{LCSS}$ | $2DD_{LCSS}$ |
|---|---|---|---|
| 50Words | 31.65 | 26.59 | 25.05 |
| Adiac | 97.19 | 85.68 | 57.54 |
| Beef | 56.67 | 46.67 | 46.67 |
| Car | 56.67 | 18.33 | 16.67 |
| CBF | 4.00 | 1.22 | 1.22 |
| ChlorineConcentration | 61.54 | 49.77 | 43.85 |
| CinC_ECG_torso | 7.10 | 7.10 | 7.10 |
| Coffee | 50.00 | 10.71 | 10.71 |
| Cricket_X | 25.90 | 25.90 | 25.90 |
| Cricket_Y | 21.28 | 18.72 | 18.72 |
| Cricket_Z | 24.36 | 24.10 | 24.87 |
| DiatomSizeReduction | 69.93 | 11.76 | 11.76 |
| ECG200 | 12.00 | 11.00 | 13.00 |
| ECGFiveDays | 5.69 | 5.57 | 5.57 |
| Face (all) | 24.38 | 22.43 | 20.18 |
| Face (four) | 18.18 | 18.18 | 18.18 |
| FacesUCR | 10.00 | 8.34 | 8.54 |
| Fish | 85.14 | 8.00 | 5.71 |
| Gun-Point | 26.67 | 6.00 | 3.33 |
| Haptics | 69.16 | 68.18 | 68.18 |
| InlineSkate | 77.82 | 61.45 | 51.64 |
| ItalyPowerDemand | 20.80 | 5.73 | 6.71 |
| Lighting-2 | 18.03 | 18.03 | 18.03 |
| Lighting-7 | 42.47 | 46.58 | 45.21 |
| MALLAT | 45.88 | 9.00 | 6.01 |
| MedicalImages | 33.42 | 33.16 | 34.47 |
| MoteStrain | 13.50 | 15.18 | 15.18 |
| Non-Invasive Thorax1 | 85.80 | 37.25 | 25.60 |
| Non-Invasive Thorax2 | 74.66 | 21.48 | 17.46 |
| OliveOil | 83.33 | 83.33 | 83.33 |
| OSU Leaf | 37.19 | 17.36 | 14.05 |
| Plane | 19.05 | 0.00 | 0.00 |
| SonyAIBORobot Surface | 29.12 | 29.12 | 13.31 |
| SonyAIBORobot SurfaceII | 16.37 | 16.37 | 14.27 |
| StarLightCurves | 17.28 | 5.63 | 3.91 |
| Swedish Leaf | 70.88 | 16.48 | 10.56 |
| Symbols | 20.70 | 3.82 | 3.22 |
| Synthetic Control | 6.00 | 6.00 | 6.00 |
| Trace | 26.00 | 4.00 | 4.00 |
| Two Patterns | 0.08 | 0.08 | 0.08 |
| TwoLeadECG | 48.46 | 18.44 | 6.94 |
| uWaveGestureLibrary_X | 32.69 | 20.49 | 20.02 |
| uWaveGestureLibrary_Y | 40.82 | 28.64 | 28.31 |
| uWaveGestureLibrary_Z | 37.52 | 27.50 | 25.77 |
| Wafer | 0.49 | 0.49 | 0.49 |
| WordsSynonyms | 33.07 | 27.43 | 27.90 |
| Yoga | 40.40 | 15.03 | 12.33 |

most data sets. This is especially clearly seen for the mean of relative errors presented in Table 3.

The average relative error reduction for all data sets is equal to 33.21% for $DD_{LCSS}$ and 37.72% for $2DD_{LCSS}$, compared with LCSS. A graphical comparison of methods is presented in Fig. 4. We see that the new methods are clearly superior to the LCSS dissimilarity measure on most of the examined data sets. Additionally we can see that the $2DD_{LCSS}$ method outperforms $DD_{LCSS}$ (with an average relative error reduction for all data sets equal to 9.72%).

It is not very useful to have a method that will perform well only on some problems, unless we can tell in advance on which problems it will be better. We must prove that we can predict ahead of time when our method will have better performance. Batista et al. [3] proposed a way of doing this. We can examine the accuracy of the compared methods by looking only at the training data set, and use these results to choose an algorithm to use to classify the testing data set. We measure the accuracy of the examined methods on the training data set using leaving-one-out cross-validation, and calculate the expected change in accuracy as

$$\frac{\text{accuracy } 2DD_{LCSS}}{\text{accuracy } DD_{LCSS}}.$$

**Table 3**
Average relative testing error rates (in %) on all data sets.

| | $\frac{DD_{LCSS}-LCSS}{LCSS}$ | $\frac{2DD_{LCSS}-LCSS}{LCSS}$ | $\frac{2DD_{LCSS}-DD_{LCSS}}{DD_{LCSS}}$ |
|---|---|---|---|
| Mean | −33.21 | −37.72 | −9.72 |

Values greater than one indicate that we expect $2DD_{LCSS}$ to outperform $DD_{LCSS}$ on the given data set. In the same way we measure the actual change in accuracy using the testing data set. The results are shown in Fig. 5 (we removed the point for the data set *Adiac* because of the scaling of the figure – it lies in the far top right of the diagram).

We can mark regions of the diagrams with the familiar labels:

- **TP** In this region we correctly predicted that $2DD_{LCSS}$ would improve accuracy.
- **TN** In this region we correctly predicted that $2DD_{LCSS}$ would decrease accuracy.
- **FN** In this region we incorrectly predicted that $2DD_{LCSS}$ would decrease accuracy.

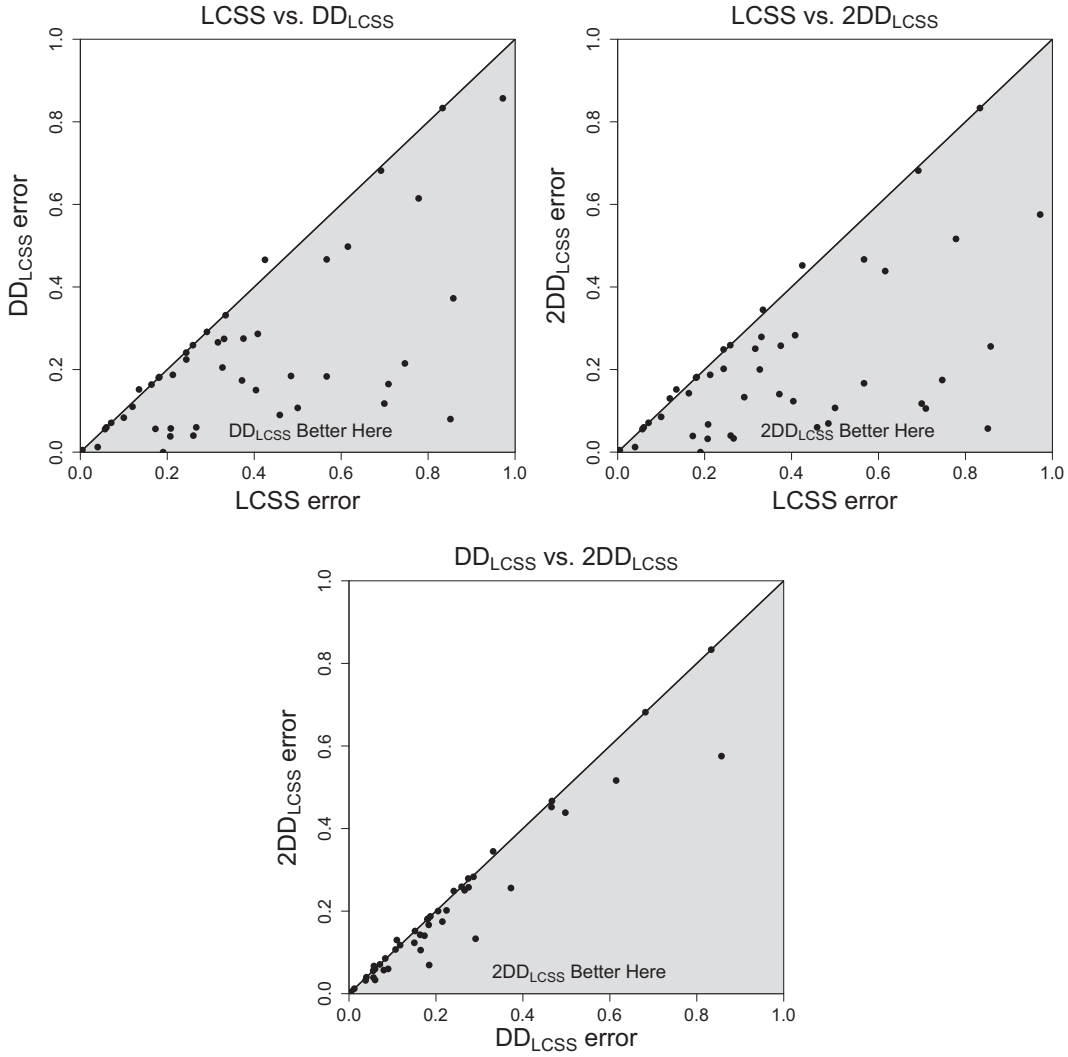


**Fig. 4.** Comparison of test errors.

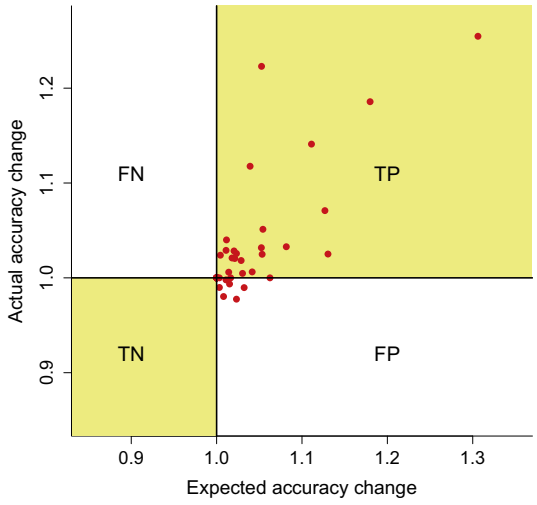


**Fig. 5.** Expected change in accuracy calculated on a training data set versus actual change in accuracy on a testing data set.

**FP** This region is the only truly bad case. Data points falling in this region represent data sets where we thought we could improve accuracy, but in fact did not.

Because $2DD_{LCSS}$ is an extension of $DD_{LCSS}$, the error rate on the training data set is always no worse than the error rate for $DD_{LCSS}$. Therefore the left part of Fig. 5 is empty. The vast majority of points lie in the TP area. This means that we are able to predict quite well when the method can improve the quality of classification. Of course, there are data sets for which the methods behave similarly: these lie near the point (1,1). In general, cases near the point (1,1) are not of special interest, since they represent marginal change in accuracy.

### 4.1. Statistical comparison of methods

To identify differences between the methods, we present a detailed statistical comparison. We test the null hypothesis that all classifiers perform the same and the observed differences are merely random. We used the Iman and Davenport [16] test, which is a nonparametric equivalent of ANOVA. Let $R_{ij}$ be the rank of the $j$th of $K$ classifiers on the $i$th of $N$ data sets and $R_j = \frac{1}{N}\sum_{i=1}^{N} R_{ij}$. The test compares the mean ranks of classifiers and is based on the statistic

$$S' = \frac{(N-1)S}{N(K-1)-S}, \tag{5}$$

where

$$S = \frac{12N}{K(K+1)}\sum_{i=1}^{K} R_i^2 - 3N(K+1) \quad (6)$$

is the Friedman statistic, which has the $F$ distribution with $K-1$ and $(K-1)(N-1)$ degrees of freedom. The $p$-value from this test is equal to 4.4E−8. We can therefore proceed with the post hoc tests in order to detect significant pairwise differences among all of the classifiers. A set of pairwise comparisons can be associated with a set of hypotheses. Any of the post hoc tests which can be applied to non-parametric tests work over a family of hypotheses. The test statistic for comparing the $i$th and $j$th classifiers is

$$Z = \frac{R_i - R_j}{\sqrt{\frac{K(K+1)}{6N}}}.$$

This statistic is asymptotically normal with zero mean and unit variance.

When comparing multiple algorithms, to retain an overall significance level $\alpha$, one has to adjust the value of $\alpha$ for each post hoc comparison. There are various methods for this. A simple method is to use Bonferroni correction. There are $m = K(K-1)/2$ comparisons, therefore Bonferroni correction sets the significance level of each comparison to $\alpha/m$. Demšar [8] recommends the procedure of Nemenyi [22] which is based on this correction. Garcia and Herrera [12] explain and compare the use of various correction algorithms. They show that although it requires intensive computation, the dynamic procedure of Bergmann and Hommel [4] is the most powerful. This procedure is based on the idea of finding all elementary hypotheses which cannot be rejected. To formulate it, we need the following definition:

An index set $I \subseteq \{1,2,\ldots,m\}$ is called *exhaustive* if exactly all $H_j$, $j \in I$, could be true.

Under this definition, the Bergmann–Hommel procedure works as follows: Reject all $H_j$ with $j \notin A$, where the *acceptance set*

$$A = \bigcup \{I : \ I \text{ exhaustive}, \ \min\{P_i : i \in I\} > \alpha/|I|\}$$

is the index set of null hypotheses which are retained. For this procedure, one has to check for each subset $I$ of $\{1,2,\ldots,m\}$ whether $I$ is exhaustive, which leads to intensive computation. Bergmann and Hommel [4] proposed a rapid algorithm which allows a substantial reduction in computing time.

The results of multiple comparisons are given in Tables 4 and 5. Those classifiers that are connected by a sequence of stars have average ranks that are not significantly different from one another. Finally we obtained two homogeneous disjoint groups of classifiers: $\{2DD_{LCSS}, DD_{LCSS}\}$ and $\{LCSS\}$. The best classifiers are in the first group.

### 4.2. Speed comparison of the investigated methods

Table 6 contains information about the computation time of particular elements during the training phase. The columns give first the times required to determine the dissimilarity matrices (3 columns), followed by the times required for the selection of parameters, for the methods $DD_{LCSS}$ and $2DD_{LCSS}$ respectively. The chosen estimation method is the cross-validation (leave-one-out) method, which is executed on the whole training set for all parameters – more strictly, for every parameter from a finite subset of the set of parameters. Therefore, the precision of the method depends on the size (density) of this subset. An important question, then, is how the number of parameters influences the time of computation. The structure of the proposed dissimilarity measures allows the development of an algorithm whose computation time depends more on the computation time of the base dissimilarity measure than on the number of parameters. Therefore, especially for base dissimilarity measures with higher computational complexity (LCSS or DTW, for example), we can examine a relatively large subset of parameters without substantial increase in computation

**Table 4**
$p$-Values in the Bergmann–Hommel post hoc test.

| $i$ | Hypothesis | $p$-Value |
|---|---|---|
| 1 | LCSS vs. $2DD_{LCSS}$ | 8.9E−7 |
| 2 | LCSS vs. $DD_{LCSS}$ | 5.8E−5 |
| 3 | $DD_{LCSS}$ vs. $2DD_{LCSS}$ | 0.279 |

**Table 5**
Results of the Bergmann–Hommel post hoc test.

| Procedure | Ranks mean | | |
|---|---|---|---|
| $2DD_{LCSS}$ | 1.57 | * | |
| $DD_{LCSS}$ | 1.80 | * | |
| LCSS | 2.63 | | * |

**Table 6**
Running times (in seconds) for the examined methods.

| | Dissimilarity matrix | | | Parameters | |
|---|---|---|---|---|---|
| | LCSS | $\nabla$LCSS | $\nabla^2$LCSS | $DD_{LCSS}$ | $2DD_{LCSS}$ |
| 50Words | 25 | 25 | 26 | 0 | 5 |
| Adiac | 8 | 8 | 9 | 0 | 4 |
| Beef | 0 | 0 | 0 | 0 | 0 |
| Car | 2 | 2 | 2 | 0 | 0 |
| CBF | 0 | 0 | 0 | 0 | 0 |
| ChlorineConcentration | 11 | 11 | 11 | 0 | 6 |
| CinC_ECG_torso | 7 | 7 | 8 | 0 | 0 |
| Coffee | 0 | 0 | 0 | 0 | 0 |
| Cricket_X | 24 | 24 | 25 | 0 | 4 |
| Cricket_Y | 24 | 24 | 25 | 0 | 4 |
| Cricket_Z | 24 | 24 | 25 | 0 | 4 |
| DiatomSizeReduction | 0 | 0 | 0 | 0 | 0 |
| ECG200 | 0 | 0 | 0 | 0 | 0 |
| ECGFiveDays | 0 | 0 | 0 | 0 | 0 |
| Face (all) | 8 | 8 | 8 | 0 | 8 |
| Face (four) | 0 | 0 | 0 | 0 | 0 |
| FacesUCR | 1 | 1 | 1 | 0 | 1 |
| Fish | 12 | 14 | 13 | 0 | 0 |
| Gun-Point | 0 | 0 | 0 | 0 | 0 |
| Haptics | 49 | 50 | 49 | 0 | 0 |
| InlineSkate | 57 | 67 | 69 | 0 | 0 |
| ItalyPowerDemand | 0 | 0 | 0 | 0 | 0 |
| Lighting-2 | 2 | 2 | 2 | 0 | 0 |
| Lighting-7 | 1 | 1 | 1 | 0 | 0 |
| MALLAT | 6 | 5 | 5 | 0 | 0 |
| MedicalImages | 2 | 2 | 2 | 0 | 4 |
| MoteStrain | 0 | 0 | 0 | 0 | 0 |
| Non-Invasive Thorax1 | 2780 | 2696 | 2880 | 4 | 89 |
| Non-Invasive Thorax2 | 2780 | 2696 | 2880 | 4 | 89 |
| OliveOil | 0 | 0 | 0 | 0 | 0 |
| OSU Leaf | 13 | 14 | 15 | 0 | 1 |
| Plane | 0 | 0 | 0 | 0 | 0 |
| SonyAIBORobot Surface | 0 | 0 | 0 | 0 | 0 |
| SonyAIBORobot SurfaceII | 0 | 0 | 0 | 0 | 0 |
| StarLightCurves | 1635 | 1586 | 1650 | 1 | 27 |
| Swedish Leaf | 6 | 6 | 6 | 0 | 6 |
| Symbols | 0 | 0 | 0 | 0 | 0 |
| Synthetic Control | 0 | 0 | 0 | 0 | 0 |
| Trace | 1 | 1 | 1 | 0 | 0 |
| Two Patterns | 23 | 22 | 23 | 1 | 27 |
| TwoLeadECG | 0 | 0 | 0 | 0 | 0 |
| uWaveGestureLibrary_X | 129 | 131 | 132 | 1 | 22 |
| uWaveGestureLibrary_Y | 129 | 131 | 132 | 1 | 22 |
| uWaveGestureLibrary_Z | 129 | 131 | 132 | 1 | 22 |
| Wafer | 37 | 35 | 35 | 1 | 27 |
| WordsSynonyms | 9 | 10 | 10 | 0 | 2 |
| Yoga | 29 | 30 | 31 | 0 | 2 |
| Mean | 169 | 165 | 175 | 0 | 8 |

time. This is possible because the new dissimilarity measures ($DD_{LCSS}$ and $2DD_{LCSS}$)do not share parameters with the base dissimilarity measures (LCSS). We observe that the procedure for selecting the parameters does in fact run very fast. The greatest computational load is associated with the determination of the dissimilarity matrices. In the case of the $2DD_{LCSS}$ method, as many as three such matrices are required. For a large number of observations, this may become computationally quite expensive. As expected, the times required to determine the particular matrices are very similar for all of the methods. It should be noted that in the case of the *Non-Invasive Thorax1* and *Non-Invasive Thorax2* data sets, which require the longest computation time, the time for selecting parameters accounts for only 1.03% of the total training time under the $2DD_{LCSS}$ method and 0.07% under the $DD_{LCSS}$ method. We also note that for smaller sets the entire procedure is extremely fast, and is often virtually instantaneous. Even for the largest datasets, the time required to determine a single dissimilarity matrix never exceeds 50 minutes, and the parameter selection phase does not take longer than 1.5 minutes in the worst case.

## 5. Conclusions

In this paper we have introduced and studied new time series dissimilarity measures based on derivatives. We used these measures to classify time series in conjunction with the 1NN method. Our studies showed that these methods give very good results. Our measures are superior to the LCSS. The proposed methods, thanks to a parametrical approach, make it possible to choose an appropriate model for any data set. The experiments that we have conducted justify the power and usefulness of our methods, especially $2DD_{LCSS}$. In general, when there are outliers in the data, use of the $2DD_{LCSS}$ method can be recommended. In a situation where computation time is significant it is also possible to use the $DD_{LCSS}$ method, which is only slightly less efficient than $2DD_{LCSS}$ (giving larger classification error) but is less computationally complex (by about one-third). We do not recommend using the pure LCSS method.

## Acknowledgments

The authors wish to thank the editor and the two referees for their comments, which have helped to improve the presentation of this paper.

## References

[1] R. Agrawal, C. Faloutsos, A. Swami, Efficient similarity search in sequence databases. in: 4th International Conference on Foundations of Data Organization and Algorithms, Springer Verlag, 1993 pp. 69–84.
[2] R. Agrawal, K. Lin, S. Sawhney, K. Shim, Fast similarity search in the presence of noise, scaling and translation in time-series databases, in: Proceedings of International Conference on Very Large Databases (VLDB'95), Zurich, Switzerland, 1995, pp. 490–501.
[3] G. Batista, X. Wang, E. Keogh, A complexity-invariant distance measure for time series, in: 11th SIAM International Conference on Data Mining (SDM'2011), Mesa, USA, 2011, pp. 699–710.
[4] G. Bergmann, G. Hommel, Improvements of general multiple test procedures for redundant systems of hypotheses, in: P. Bauer, G. Hommel, E. Sonnemann (Eds.), Multiple Hypotheses Testing, Springer, 1988, pp. 110–115.
[5] D.J. Berndt, J. Clifford, Using dynamic time warping to find patterns in time series, in: AAAI Workshop on Knowledge Discovery in Databases (KDD'94), Seattle, Washington, USA, 1994, pp. 229–248.
[6] A. Carlier, Factor analysis of evolution and cluster methods on trajectories, in: Proceedings of COMPSTAT'86, Physica-Verlag, Wien, 1986, pp. 140–145.
[7] R. Coppi, P. D'Urso, P. Giordani, A Fuzzy clustering model for multivariate spatial time series, J. Classification 27 (2010) 54–88.
[8] J. Demšar, Statistical Comparisons of Classifiers Over Multiple Data Sets, J. Mach. Learn. Res. 7 (2006) 1–30.
[9] H. Ding, G. Trajcevski, P. Scheuermann, X. Wang, E. Keogh, Querying and mining of time series data: experimental comparison of representations and distance measures, in: Proceedings of the 34th International Conference on Very Large Data, Bases, 2008, pp. 1542–1552.
[10] P. D'Urso, M. Vichi, Dissimilarities between trajectories of a three-way longitudinal data set, in: A. Rizzi, M. Vichi, H.H. Bock (Eds.), Advances in Data Science and Classification, Studies in Classification, Data Analysis, and Knowledge Organization, Springer-Verlag, Heidelberg, 1998, pp. 585–592.
[11] C. Faloutsos, M. Ranganathan, Y. Manolopoulos, Fast subsequence matching in timeseries databases, ACM SIGMOD Record 23 (1994) 419–429.
[12] S. Garcia, F. Herrera, An extension on statistical comparisons of classifiers over multiple data sets for all pairwise comparisons, J. Mach. Learn. Res. 9 (2008) 2677–2694.
[13] T. Górecki, Two parametrical derivative dynamic time warping, in: J. Pociecha, R. Decker (Eds.), Data analysis methods and its applications, C.H. Beck, 2012, pp. 31–40.
[14] T. Górecki, M. Łuczak, Using derivatives in time series classification, Data Min. Knowl. Discov. 26 (2) (2013) 310–331.
[15] T. Górecki, M. Łuczak, First and second derivative in time series classification using DTW, Comm. Statist. Simulation Comput. (2013), http://dx.doi.org/10.1080/03610918.2013.775296.
[16] R. Iman, J. Davenport, Approximations of the critical region of the freidman statistic, Comm. Statist. Theory Methods 9 (6) (1980) 571–595.
[17] E. Keogh, M. Pazzani, Dynamic time warping with higher order features, in: First SIAM International Conference on Data Mining (SDM'2001), Chicago, USA, 2001a.
[18] E. Keogh, K. Chakrabarti, M. Pazzani, S. Mehrotra, Dimensionality reduction for fast similarity search in large time series databases, Knowl. Inf. Syst. 3 (2001) 263–286.
[19] E. Keogh, Q. Zhu, B. Hu, Y. Hao, X. Xi, L. Wei, C.A. Ratanamahatana, The UCR Time Series Classification/Clustering, 2011. Homepage: <http://www.cs.ucr.edu/eamonn/time_series_data>.
[20] K. Kosmelj, Means for analysis of space-time relationships, in: Proceedings of COMPSTAT'84, Physica-Verlag, Wien, 1984, pp. 511–516.
[21] V. Kurbalija, M. Radovanovi, Z. Geler, M. Ivanovi, The influence of global constraints on similarity measuresfor time-series databases, Knowl. Based Syst. (2013). http://dx.doi.org/10.1016/j.knosys.2013.10.021 .
[22] P. Nemenyi, Distribution-free Multiple Comparisons, PhD thesis, Princeton University, 1963.
[23] M. Radovanović, A. Nanopoulos, M. Ivanović, Time-series classification in many intrinsic dimensions, in: 10th SIAM International Conference on Data Mining (SDM'2010), Columbus, USA, 2010, pp. 677–688.
[24] C.A. Ratanamahatana, J. Lin, D. Gunopulos, E. Keogh, M. Vlachos, G. Das, Mining time series data, in: Data Mining and Knowledge Discovery Handbook, second ed., Springer, 2010, pp. 1049–1077.
[25] C.A. Ratanamahatana, E. Keogh, Three myths about dynamic time warping data mining, in: 5th SIAM International Conference on Data Mining (SDM'05), Newport, USA, 2005, pp. 506–510.
[26] N. Tomašev, D. Mladenić, Nearest neighbor voting in high dimensional data: learning from past occurrences, Comput. Sci. Inf. Syst. 9 (2012) 691–712.
[27] F. van der Heijden, R.P.W. Duin, D. de Ridder, D.M.J. Tax, Classification, Parameter Estimation and State Estimation, Wiley, 2004.
[28] M. Vlachos, D. Gunopoulos, G. Kollios, Discovering similar multidimensional trajectories, in: Proceedings of the 18th International Conference on Data Engineering (ICDE'02), San Jose, USA, 2002, pp. 673–684.
[29] B.K. Yi, H. Jagadish, C. Faloutsos, Efficient retrieval of similar time sequences under time warping, in: Proceedings of the 14th International Conference on Data Engineering (ICDE'98), Orlando, USA, 1998, pp. 23–27.